Binary Number Conversion

Decimal to Binary Conversion:

By repeatedly dividing a number by 2 and recording the result, decimal values can be transformed into binary.

Example: Converting 13 to its binary equivalent:

- Start with the decimal number 13.
- Divide the number by 2 and record the remainder.
- Repeat the division with the quotient until the number becomes 0.

13 / 2 = 6 remainder 1

6 / 2 = 3 remainder 0

3 / 2 = 1 remainder 1

1 / 2 = 0 remainder 1

To obtain the binary equivalent of 13, read the remainders from bottom to top: **1101**.

So, the binary equivalent of 13 is 1101.

Binary to Decimal Conversion:

Converting a binary number back to its decimal equivalent involves a reverse process.

Example: Converting 1101 to its decimal equivalent:

- Start from the rightmost bit (least significant bit).
- Each bit is multiplied by 2 raised to the power of its position index.

1 * 2^0 = 1

0 * 2^1 = 0

1 * 2^2 = 4

1 * 2^3 = 8

Sum = 1 + 0 + 4 + 8 = **13**.

Hence, the decimal equivalent of the binary number 1101 is 13.

Understanding One's Complement and Two's Complement

One's Complement

The one's complement of a binary number is obtained by flipping all the bits.

Example: The one's complement of 13 (binary 1101):

Binary of 13 : 0000 1101

One's Complement : 1111 0010

Two's Complement

The two's complement is obtained by taking the one's complement of a number and adding 1.

Example: The two's complement of 13 (binary 1101):

One's Complement : 1111 0010

Add 1 : 1111 0011

Steps to Write Binary of a Negative Integer

Suppose we want binary of: -5

Step 1: Write positive binary

Binary of 5:

$$5_{10} = 00000101_2$$

(assuming 8 bits)

Step 2: Take 1's Complement

Invert all bits:

00000101
11111010

Step 3: Add 1

11111010
+ 1

11111011

So:

$$-5_{10} = 11111011_2$$

Shortcut Rule

For negative numbers:

Negative Binary =
(Positive Binary Inverted) + 1

This is called:

2's Complement

Why Java Uses This?

2's complement makes arithmetic easier for CPU.

Example:

5 + (-5) = 0

Binary:

00000101
11111011

00000000 (carry ignored)

Important Thing in Java

Java integers are stored in:

Type	Bits
byte	8
short	16
int	32

Type	Bits
long	64

So Java stores -5 as a 32-bit binary:

11111111 11111111 11111111 11111011

Example:

System.out.println(Integer.toBinaryString(-5));

Output:

11111111111111111111111111111011

Because Java prints full 32-bit 2's complement representation.

Bitwise Operators

AND Operator (&)

If both corresponding bits are 1, the resulting bit is 1; otherwise, it is 0.

13: 1101

7: 0111

& : 0101 → 5

OR Operator (|)

If either corresponding bit is 1, the resulting bit is 1.

13: 1101

7: 0111

| : 1111 → 15

XOR Operator (^)

If bits differ, the result is 1; if the same, result is 0.

Even number of 1's = 0

Odd number of 1's = 1

13: 1101

7: 0111

^ : 1010 → 10

NOT Operator (~)

- Flips all bits of the number.
- If sign(msb) bit is '1' take it's 2's complement.
- Else stop it.

5: 0000 0101

~5: 1111 1010 → -6 (in two's complement)

Shift Operators

Right Shift (>>): Shifts bits to the right, fills left with 0s.

13 >> 1 = 0110 → 6

num >> k = num/2^k

Left Shift (<<): Shifts bits to the left, fills right with 0s.

13 << 1 = 11010 → 26

num << k = num * 2^k

Swapping Two Numbers Without a Third Variable

- Use the XOR (^) operator to swap two integers without a temporary variable.
- Set a = a ^ b. This updates a to store the XOR of both values.
- Set b = a ^ b. This changes b to the original value of a.
- Set a = a ^ b. This updates a to the original value of b.
- This technique works due to XOR properties: x ^ x = 0 and x ^ 0 = x.

Checking if the i-th Bit is Set: return true if i-th bit is 1.

(1 << i) & num → set if result ≠ 0

(num >> i) & 1 → set if result ≠ 0

- Use bit masking to isolate the i-th bit in the binary representation of the number.
- By shifting 1 to the left i times, you create a mask where only the i-th bit is set.
- Perform a bitwise AND between the number and the mask to check if that specific bit is active.
- If the result is non-zero, the i-th bit is set to 1; otherwise, it is 0.

Setting the i-th Bit: 0 to 1 & 1 to 1.

num | (1 << i)

Clearing the i-th Bit: 1 to 0 & 0 to 0.

num & ~(1 << i)

Toggling the i-th Bit: 1 to 0 & 0 to 1.

num ^ (1 << i)

Remove last set bit(Right most): '1' to 0

num & (num-1)

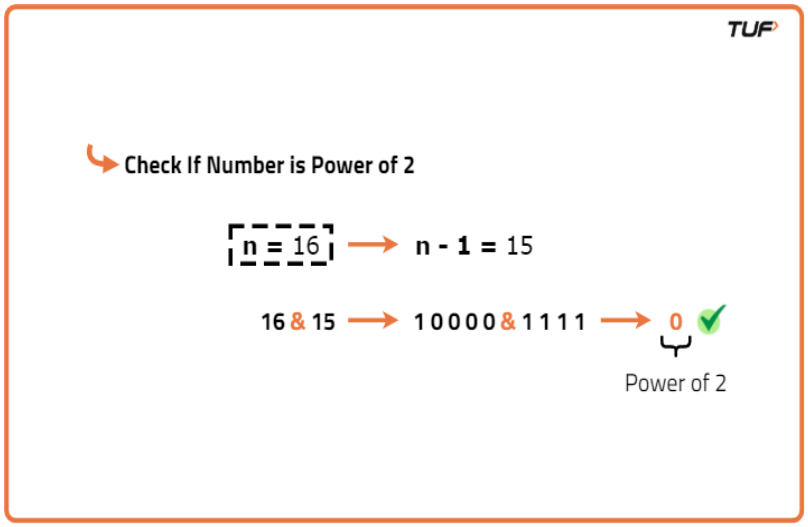
Check given number is a power of two or not

num & (num-1) == 0 then yes else no.

Algorithm

- Power of two numbers have exactly one bit set in their binary form.
- Subtracting one flips all bits after the set bit, creating no overlap with the original number.
- A bitwise AND between the number and one less than itself will be zero only for powers of two.
- This property allows for a fast check without looping or dividing.

Dry Run:



Count set bits that is No of 1's

Method 1 Intuition — Check Every Bit One by One

```
while (n > 0){
   count += n & 1;
   n = n >> 1;
}
```

Core Idea

A binary number is made of 0s and 1s.

Example:

13 = 1101

We want to count how many 1s are present.

How it works

Step 1 → Check last bit using n & 1

n & 1

1 in binary:

0001

AND operation with 1 checks only the **last bit**.

Example:

1101
0001

0001 => 1

So:

- if last bit is 1 → result is 1
- if last bit is 0 → result is 0

That value gets added to count.

Step 2 → Right shift the number

n = n >> 1;

Right shift removes the last bit.

Example:

1101 -> 110

Now again check the new last bit.

Dry Run for 13

Binary:

13 = 1101

n (binary)	n & 1	count
1101	1	1
110	0	1
11	1	2
1	1	3

Answer = 3

Intuition Summary

This method says:

“Keep checking the last bit, then remove it.”

So we inspect every bit individually.

Time Complexity:

O(number of bits)

For integer → about O(32)

Method 2 Intuition — Remove One Set Bit at a Time

```
while (n > 0){
   n = n & (n-1);
   count++;
}
```

This is a very smart bit trick.

Most Important Observation

When we do:

n & (n-1)

it removes the **rightmost set bit (1)**.

Why does this happen?

Example:

n = 13 = 1101

Now:

n-1 = 12 = 1100

AND them:

```
1101
1100
----
1100
```

See what happened?

The last 1 disappeared.

Another Example

n = 101000

Subtract 1:

100111

Now AND:

101000
100111

100000

Again, the rightmost 1 got removed.

Actual Logic

Each iteration removes exactly **one set bit**.

So:

- if number has 3 set bits → loop runs 3 times
- if number has 10 set bits → loop runs 10 times

That's why this method is faster.

Dry Run for 13

13 = 1101

Iteration 1

1101 & 1100 = 1100
count = 1

Iteration 2

1100 & 1011 = 1000
count = 2

Iteration 3

1000 & 0111 = 0000
count = 3

Loop stops.

Answer = 3

Intuition Summary

Method 2 says:

“Instead of checking every bit, directly remove set bits one by one.”

That’s why it is more optimized.

Complexity Comparison

Method	Idea	Complexity
Method 1	Check every bit	O(total bits)
Method 2	Remove set bits directly	O(number of set bits)

Given an integer n, return *true if it is a power of three. Otherwise, return false*.

An integer n is a power of three, if there exists an integer x such that n == 3^x.

1. Multiplication Approach

```
while(temp < n){
   temp *= 3;
}
```

Intuition

Keep generating powers of 3:

1, 3, 9, 27, 81...

If we reach n → true
If we cross n → false

Complexity

Time	Space
$O(\log\ n)$	O(1)

---

2. Division Approach

```
while(n % 3 == 0){
   n /= 3;
}
```

Intuition

If n is a power of 3, repeatedly dividing by 3 should finally make it 1.

Example:

81 -> 27 -> 9 -> 3 -> 1

Complexity

Time	Space
$O(\log\ n)$	O(1)

---

3. Largest Power Trick

```
return n > 0 && 1162261467 % n == 0;
```

Intuition

1162261467 is the largest power of 3 inside int range.

Every smaller power of 3 divides it perfectly.

Complexity

Time	Space
O(1)	O(1)

Given an integer n, return true *if it is possible to represent* n *as the sum of distinct powers of three.* Otherwise, return false.

An integer y is a power of three if there exists an integer x such that y == 3^x.

💡 Intuition

The problem asks:

Can the number be formed using distinct powers of 3?

Distinct means:

each power can be used at most once

Example:

12 = 9 + 3

Valid because:

3^2 + 3^1

each power used once.

Main Observation

This problem becomes very easy if we think in:

base 3 (ternary)

Because every digit in base 3 tells:

how many times a power of 3 is used

Example:

12 in base 3 = 110

Meaning:

1×9 + 1×3 + 0×1

Here every digit is:

0 or 1

So every power is used at most once → valid.

Invalid Case

Suppose a ternary digit becomes:

2

Example:

210(base 3)

This means:

2 × 3^2

So same power is used twice.

That violates the “distinct powers” condition.

Hence:

If any remainder becomes 2, answer is false.

Why % 3 and / 3 Work

- % 3 extracts current ternary digit
- / 3 removes that digit and moves to next power

Exactly same as extracting binary digits using %2 and /2.

Overall Thinking

The solution is basically checking:

“Does ternary representation contain only 0 and 1?”

If yes → valid
If digit 2 appears → invalid.

Given two integers dividend and divisor, divide two integers without using multiplication, division, and mod operator.

The integer division should truncate toward zero, which means losing its fractional part. For example, 8.345 would be truncated to 8, and -2.7335 would be truncated to -2.

Return *the quotient after dividing* dividend *by* divisor.

Note: Assume we are dealing with an environment that could only store integers within the 32-bit signed integer range: [-2^{31}, $2^{31} - 1$]. For this problem, if the quotient is strictly greater than $2^{31} - 1$, then return $2^{31} - 1$, and if the quotient is strictly less than -2^{31}, then return -2^{31}.

Intuition

Instead of subtracting divisor repeatedly:

43 - 3 - 3 - 3 ...

we subtract the largest possible multiple of divisor using bit manipulation.

Left shift helps multiply by powers of 2:

$$d \ll k = d \times 2^k$$

So we keep finding the biggest shifted divisor that fits inside the dividend and add that power of 2 to the quotient.

This makes division much faster.

Time Complexity

Outer loop reduces dividend exponentially.

Inner loop finds largest shift.

Complexity:

$$O((\log\ n)^2)$$

Space Complexity

$$O(1)$$

Interview Problems

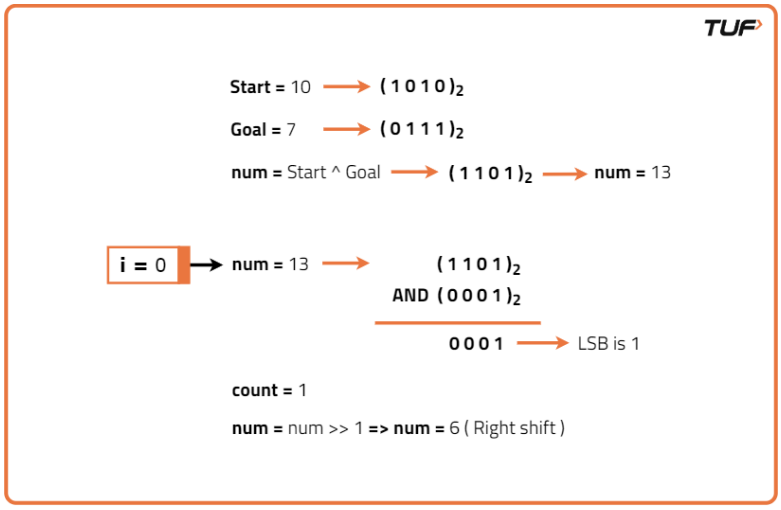
Given two integers start and goal. Flip the minimum number of bits of start integer to convert it into goal integer. A bits flip in the number val is to choose any bit in binary representation of val and flipping it from either 0 to 1 or 1 to 0.

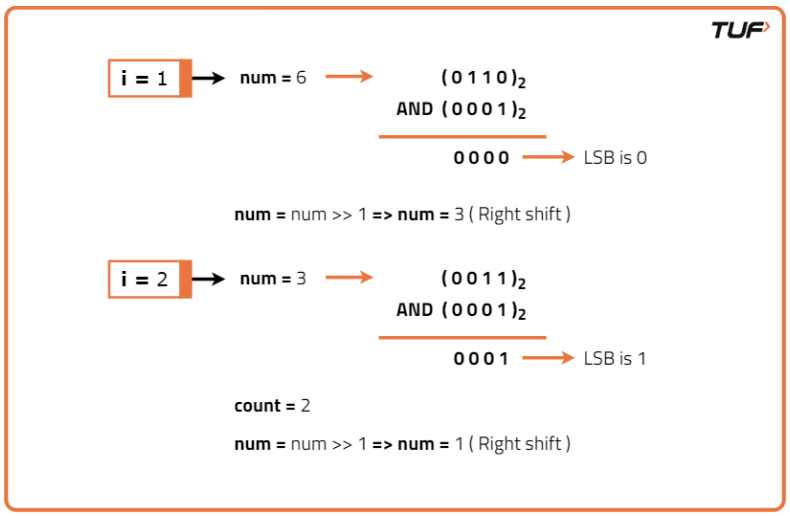
Algorithm

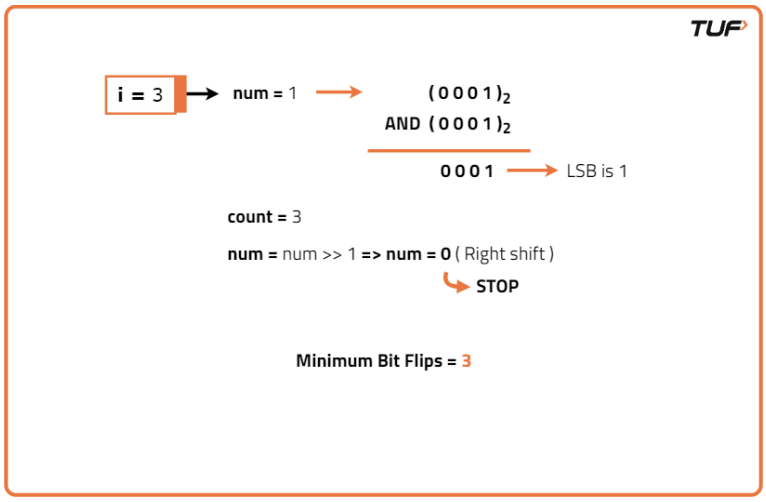
- To find how many bits differ between two numbers, compare them bit by bit.
- XOR highlights the positions where the two numbers have different bits.
- Each set bit in the XOR result represents a required flip to match the numbers.
- Count the number of set bits in the XOR result to get the total flips needed.

- This provides the minimum number of bit changes to convert one number into the other.

Dry Run:







Given an integer array nums where every element appears **three times** except for one, which appears **exactly once**. *Find the single element and return it*.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Approach-1

Store frequency of every number.

- numbers repeating 3 times → frequency becomes 3
- unique number → frequency becomes 1

Return the number whose frequency is 1.

Complexity

Complexity	Value
Time	O(n)
Space	O(n)

Approach-2

Every repeating number contributes bits in multiples of 3.

So:

- count set bits at every position
- take modulo 3
- remaining bits belong to unique number

Example

Array:

1 = 001
2 = 010
1 = 001

2 = 010
3 = 011
1 = 001
2 = 010

Count Bit Position 0

Number	Bit0
1	1
2	0
1	1
2	0
3	1
1	1
2	0

Total:

4

4 % 3 = 1

So bit0 belongs to answer.

Count Bit Position 1

Total set bits:

4

4 % 3 = 1

So bit1 also belongs to answer.

Final bits:

011 = 3

Complexity

Complexity	Value
Time	O(32*n) ≈ O(n)
Space	O(1)

Approach-3

After sorting:

[1,1,1,2,2,2,3]

Numbers repeating 3 times come together.

So:

- check groups of 3
- if group breaks, unique number found

Dry Run

Sorted array:

[1,1,1,2,2,2,3]

Check:

1 == 1
2 == 2

No mismatch.

Last remaining element:

3

Complexity

Complexity	Value
Time	O(n log n)

Complexity	Value
Space	O(1)

Approach-4

This approach is actually doing:

count of every bit modulo 3

using two bitmasks:

Variable	Stores bits whose count is
ones	count % 3 == 1
twos	count % 3 == 2

Bit state transitions:

0 occurrence -> ones -> twos -> removed

So:

- 1st occurrence → bit goes to ones
- 2nd occurrence → bit moves to twos
- 3rd occurrence → bit removed from both

Thus, bits of numbers appearing 3 times disappear automatically.

Only the unique number remains in ones.

Formula Meaning

ones = (ones ^ e) & ~twos;

- ones ^ e → toggle bits in ones
- & ~twos → remove bits already present in twos

---

twos = (twos ^ e) & ~ones;

- toggle bits in twos

- remove bits already present in ones

Dry Run

Array:

[1,2,1,2,3,1,2]

Element	ones	twos
start	0	0
1	1	0
2	3	0
1	2	1
2	0	3
3	0	0
1	1	0
2	3	0

Final:

ones = 3

Answer = 3

Complexity

Complexity	Value
Time	O(n)
Space	O(1)

Given an integer array nums, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in **any order**.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Intuition

Suppose unique numbers are:

a and b

All duplicate numbers cancel out using XOR:

x ^ x = 0

So after XOR of entire array:

temp = a ^ b

Since:

- a != b

there must be at least one bit different between them.

We find one such differing bit (rightMost set bit).

Using that bit:

- one unique number goes into first bucket
- other unique number goes into second bucket

Duplicate numbers always go into same bucket and cancel out.

Finally:

- one bucket gives first unique number
- second bucket gives second unique number

Understanding rightMost

int rightMost = (temp & (temp-1)) ^ temp;

This extracts:

rightmost set bit

Equivalent simpler formula:

int rightMost = temp & (-temp);

Why This Works

That bit is different in a and b.

Example:

a = 3 = 011
b = 5 = 101

a ^ b = 110

Rightmost set bit:

010

This bit:

- set in one number
- unset in other

So they get separated into different buckets.

Dry Run

Array:

[1,2,1,3,2,5]

Unique numbers:

3 and 5

Step 1 → XOR Entire Array

temp = 1^2^1^3^2^5

Duplicates cancel:

= 3 ^ 5

Binary:

3 = 011
5 = 101

011 ^ 101 = 110

So:

temp = 110

Step 2 → Find Rightmost Set Bit

rightMost = 010

This means:
check second bit.

Step 3 → Divide Into Buckets

Condition:

(e & rightMost) == 0

Bucket 1 (bit not set)

Number	Binary	Goes To
1	001	first
1	001	first
5	101	first

XOR:

1 ^ 1 ^ 5 = 5

Bucket 2 (bit set)

Number	Binary	Goes To
2	010	second
3	011	second
2	010	second

XOR:

2 ^ 2 ^ 3 = 3

Final Answer

[5,3]

(or [3,5])

Complexity

Complexity	Value
Time	O(n)
Space	O(1)

You are given two integers L and R, your task is to find the **XOR** of elements of the range [L, R].

Intuition

We use a property of XOR from 1 to n.

Pattern:

n % 4 == 0 -> n
n % 4 == 1 -> 1
n % 4 == 2 -> n + 1
n % 4 == 3 -> 0

So instead of calculating XOR one-by-one, we directly compute it in O(1).

Why It Works

Suppose:

XOR(1 to r) = A
XOR(1 to l-1) = B

Then:

A ^ B

removes common elements from 1 to l-1.

Because:

$$x \oplus x = 0$$

So remaining XOR is:

l ^ (l+1) ^ ... ^ r

Hence:

helper(l-1) ^ helper(r)

Approach

1. Create helper function to find XOR from 1 to n.
2. Use modulo 4 pattern.
3. Compute:

XOR(l to r) = XOR(1 to r) ^ XOR(1 to l-1)

Dry Run

Example:

l = 3
r = 7

Step 1

Find:

XOR(1 to 7)

Since:

7 % 4 = 3

helper returns:

0

Step 2

Find:

XOR(1 to 2)

Since:

2 % 4 = 2

helper returns:

2 + 1 = 3

Step 3

Final answer:

0 ^ 3 = 3

Now verify manually:

3 ^ 4 ^ 5 ^ 6 ^ 7

= 3

Correct.

Time Complexity

$$O(1)$$

because only constant operations are performed.

Space Complexity

$$O(1)$$

Given an integer array nums of unique elements, return *all possible subsets (the power set)*. The solution set must not contain duplicate subsets. Return the solution in any order.

Intuition

Every element has 2 choices:

Include it
OR
Exclude it

So for n elements:

$$2^n$$

subsets are possible.

We use binary representation to represent these choices.

Binary Representation Idea

For:

nums = [1,2,3]

we have:

n = 3

Total subsets:

2^3 = 8

Numbers from:

0 to 7

in binary:

000
001
010
011
100
101
110
111

Each bit represents whether an element is included.

Mapping

Binary Subset

000 []

001 [1]

010 [2]

011 [1,2]

100 [3]

101 [1,3]

110 [2,3]

111 [1,2,3]

Approach

1. Run loop from 0 to (1<<n)-1.
2. Every number represents one subset.
3. Check every bit using:

(i & (1<<j))

4. If bit is set, include that element in subset.
5. Add subset to result.

Why It Works

Expression:

(1 << j)

creates mask with only j-th bit set.

Example:

1 << 2 = 100

Now:

i & (1<<j)

checks whether j-th bit in i is set or not.

If result > 0:

element is included

Dry Run

Example:

nums = [1,2,3]

Take:

i = 5

Binary:

101

Check bits:

- j = 0

101 & 001 = 1

Include 1

- j = 1

101 & 010 = 0

Do not include 2

- j = 2

101 & 100 = 100

Include 3

Subset becomes:

[1,3]

Time Complexity

Outer loop:

2^n

Inner loop:

$$n$$

Total:

$$O(n \times 2^n)$$

Space Complexity

Ignoring output:

$$O(n)$$

Including result storage:

$$O(n \times 2^n)$$